

ML307C

固件升级开发指导手册

版本：V1.0.0

发布日期：2025/3/7

服务与支持

如果您有任何关于模组产品及产品手册的评论、疑问、想法，或者任何无法从本手册中找到答案的疑问，请通过以下方式联系我们。

OneMO官网： onemo10086.com

邮箱： SmartModule@cmiot.chinamobile.com

客户服务热线： 400-110-0866



中国移动
China Mobile

文档声明

注意

本手册描述的产品及其附件特性和功能，取决于当地网络设计或网络性能，同时也取决于用户预先安装的各种软件。由于当地网络运营商、ISP，或当地网络设置等原因，可能也会造成本手册中描述的全部或部分产品及其附件特性和功能未包含在您的购买或使用范围之内。

责任限制

除非合同另有约定，中移物联网有限公司对本文档内容不做任何明示或暗示的声明或保证，并且不对特定目的适销性及适用性或者任何间接的、特殊的或连带的损失承担任何责任。

在适用法律允许的范围内，在任何情况下，中移物联网有限公司均不对用户因使用本手册内容和本手册中描述的产品而引起的任何特殊的、间接的、附带的或后果性的损坏、利润损失、数据丢失、声誉和预期的节省而负责。

因使用本手册中所述的产品而引起的中移物联网有限公司对用户的最大赔偿（除在涉及人身伤害的情况中根据适用法律规定的损害赔偿外），不应超过用户为购买此产品而支付的金额。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。公司保留随时修改本手册中任何信息的权利，无需进行提前通知且不承担任何责任。

商标声明



为中国移动注册商标。

本手册和本手册描述的产品中出现的其他商标、产品名称、服务名称和公司名称，均为其各自所有者的财产。

进出口法规

出口、转口或进口本手册中描述的产品（包括但不限于产品软件和技术数据），用户应遵守相关进出口法律和法规。

隐私保护

关于我们如何保护用户的个人信息等隐私情况，请查看相关隐私政策。

操作系统更新声明

操作系统仅支持官方升级；如用户自己刷非官方系统，导致安全风险和损失由用户负责。

固件包完整性风险声明

固件仅支持官方升级；如用户自己刷非官方固件，导致安全风险和损失由用户负责。

版权所有©中移物联网有限公司。保留一切权利。

本手册中描述的产品，可能包含中移物联网有限公司及其存在的许可人享有版权的软件，除非获得相关权利人的许可，否则，非经本公司书面同意，任何单位和个人不得擅自摘抄、复制本手册内容的部分或全部，并以任何形式传播。



中国移动
China Mobile

关于文档

修订记录

版本	描述
V1.0.0	初版



中国移动
China Mobile

目录

服务与支持.....	ii
文档声明.....	iii
关于文档.....	v
1. 概述.....	7
1.1. 适用范围.....	7
1.2. 写作目的.....	7
2. 功能说明.....	8
3. API说明.....	9
4. 应用指导.....	10
4.1. 制作升级包.....	10
4.1.1. 制作固件升级包.....	10
4.2. 本地升级应用示例.....	13
5. 编程设计注意.....	28
6. 附录.....	29



中国移动
China Mobile

1. 概述

本文档介绍OpenCPU SDK的固件升级应用开发，包含固件升级函数API接口和升级包制作方法的介绍。开发者可基于相关API接口进行固件本地或远端升级开发的应用程序的编程。

1.1. 适用范围

Table 1. 适用模组

模组型号	模组子型号
ML307C	ML307C-DC-CN

1.2. 写作目的

本文档用于介绍固件升级开发功能。

本文档介绍了OpenCPU SDK下固件升级功能，旨在为用户使用SDK进行应用程序开发提供参考，用户可参照本文档和SDK中的示例程序进行应用程序编写。

2. 功能说明

本章主要介绍固件升级功能。

固件升级，指的是对模块的内嵌固件进行升级。用户固件更新后，可通过固件升级的方式更新模块内嵌的固件。

本模组支持最小系统升级、全系统差分升级、APP整包升级三种方案。关于本平台独有的最小系统升级方案，具体如下。

最小系统升级方案下，系统分为可连网的最小系统和非最小系统（用户开发的应用APP代码均属于非最小系统），固件升级时两个系统均需要升级。升级时首先下载最小系统差分文件，下载完成后重启进入升级。完成最小系统的升级后重启运行的是升级后的可连网的最小系统，然后再下载非最小系统升级文件，该文件是全包非差分文件，升级时直接覆盖非最小系统区域，下载完成后校验，校验通过后整个升级完成。

最小系统升级方案下，包括最小系统升级和非最小系统升级。使用升级包制作工具制作出的升级包中包含最小系统和非最小系统升级包的内容。

由于芯片平台特性，模组完成一次上述最小系统升级至少需要重启四次。若升级过程中任一阶段的任务完成，模组会自动重启，重启后执行下一阶段的任务。若升级过程中任一阶段的任务超时未完成，模组会自动重启，重启后重新尝试完成本阶段的任务，用户可通过`cm_fota_set_reboot_time()`接口设置超时（重启）时间（网络较差或升级包较大时，尝试重启的超时时间不宜设置过短）。

3. API说明

SDK提供一套完整的固件升级用户编程API。

最小系统升级方案下，固件升级接口定义详见SDK中cm_fota.h文件。

全系统差分升级和APP整包升级方案下，固件升级接口定义详见SDK中cm_ota.h文件。



中国移动
China Mobile

4. 应用指导

本章介绍了常见的应用示例。

4.1. 制作升级包

本节主要介绍升级包制作相关的操作流程和工具。

4.1.1. 制作固件升级包

在制作差分包之前，需要先通过编译获得升级前后的版本包，即模组当前版本和目标升级版本。接着使用升级包制作工具，生成升级包。¹

4.1.1.1. 最小系统升级方案

操作步骤

最小系统升级方案下，执行以下步骤完成升级包制作。

1. 将固件版本包放入制作工具adiff.exe的根目录。

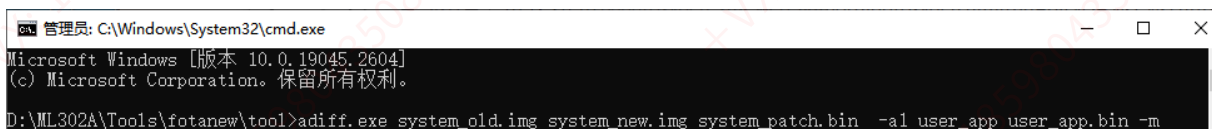
参考下图流程，执行解压、拷贝、重命名操作获取升级前后版本包。



2. 进入adiff.exe目录执行命令，得到升级包。

进入目录执行如下命令，生成升级文件system_patch.bin。

```
adiff.exe system_old.img system_new.img system_patch.bin -a1 user_app user_app.bin -m
```



4.1.1.2. 全系统差分升级方案

操作步骤

全系统差分升级方案下，执行以下步骤完成升级包制作。

1. 制作升级包过程中，若命令执行错误，建议手动输入命令。

1. 将固件版本包放入制作工具adiff.exe的根目录。



2. 进入adiff.exe目录执行命令，得到升级包。

```
adiff.exe -p system_old.img system_new.img system_patch.bin -a1 user_app user_app_old.bin user_app_new.bin -l fsall -s 25000
```

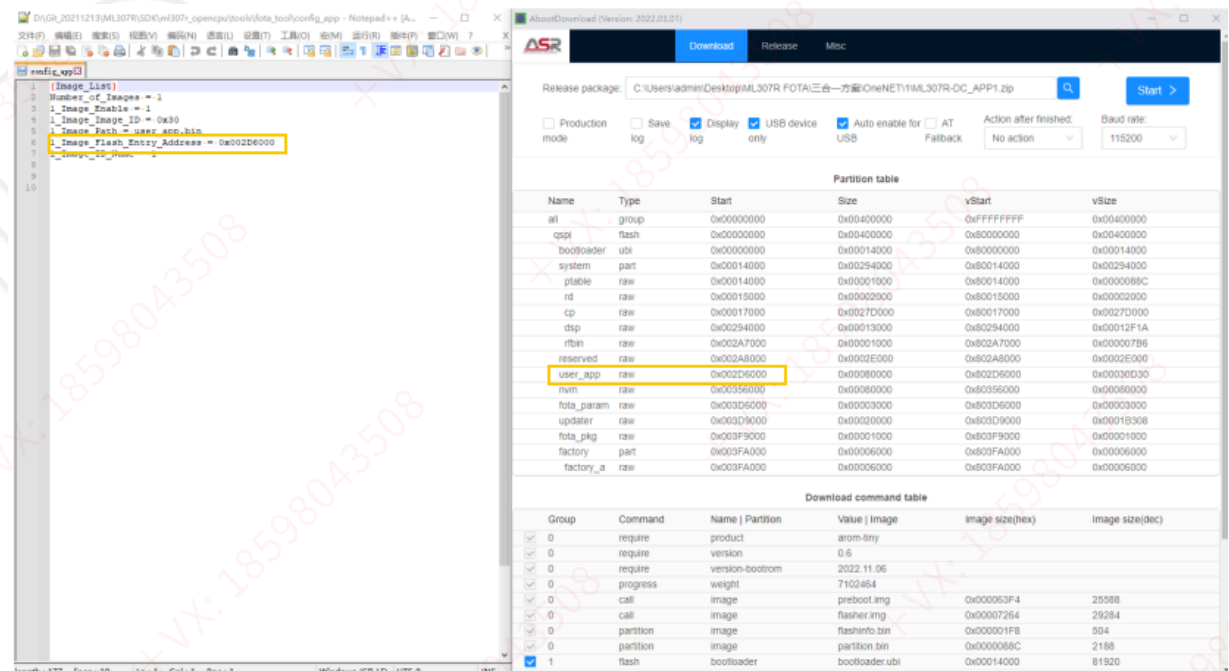


4.1.1.3. APP整包升级方案

操作步骤

APP整包升级方案下，执行以下步骤完成升级包制作。

1. 确认升级包制作工具中config_app文件内1_Image_Flash_Entry_Address与固件包中user_app起始地址一致。固件包中user_app起始地址可通过固件下载工具获取。



2. 将固件版本包放入制作工具fota_tool\A目录。
3. 解压目标升级版本，拷贝user_app.bin至fota_tool\A目录。
4. 进入FBFMake_CF_V1.6-150.exe目录（与adiff目录一致）执行命令，得到升级包。

```
FBFMake_CF_V1.6-150.exe -o system_patch.bin -f config_app -a a -b a
```



 Note:

升级前后版本号管控应由用户实现；

不同SDK版本使用的升级包制作工具和制作方法可能存在差异，请联系中移物联网FAE获取对应SDK版本的固件升级包制作工具（SDK中tools\fota_tool路径下）和工具使用方法；

生成升级文件后，可上传升级文件至升级服务器以供后续下载升级（相关注意事项见【编程设计注意】章节）。



中国移动
China Mobile

4.2. 本地升级应用示例

本节介绍了常见的应用示例。

最小系统升级方案

1. 最小系统升级方案下，HTTP方式下载升级包数据并升级。

```
cm_fota_res_callback_register((cm_fota_result_callback)__cm_fota_cb);
cm_fota_set_url("http://xxx.com:8080/system_patch.bin"); //仅作为示例，url不可使用。
cm_fota_exec_upgrade();
```

2. 最小系统升级方案下，OneNET FOTA（HTTP协议）下载升级包数据并升级（本文档仅摘录部分代码，完整代码参考cm_demo_fota.c中源码）。

填写OneNET产品和设备信息（用于与OneNET平台交互）示例如下：

```
/*
  产品ID，平台创建产品后由平台分配的ID（https://open.iot.10086.cn/console/product/own）。此处为示例数据，实际使用
  须据实修改 */
static char OneNET_pid[16] = "07av4iD9CC";

/*
  用户ID，平台网页中用户中心-账户信息（https://open.iot.10086.cn/account/profile）中查询到的用户ID。此处为示例数
  据，实际使用须据实修改 */
static char OneNET_userid[16] = "148865";

/*
  设备名称，平台添加设备时添加设备的设备名称（https://open.iot.10086.cn/console/device/manage/devs）。此处为示例
  数据，实际使用须据实修改 */
static char OneNET_devname[16] = "ML307A-DSLN";

/*
  用户Accesskey，平台网页中用户中心-访问权限（https://open.iot.10086.cn/account/access）中查询到的用户
  Accesskey。此处为示例数据，实际使用须据实修改 */
static char OneNET_userkey[128]
= "xxxxxxx9acyjz4upifwnvbwuETqplZ0DEJ+9jTj2jWD0GzrDSrLu7/oqLmKIZk18SUPu6vQZDsaOrwtPethioA==";

/* 模组OneNET软件版本号，平台添加差分升级包时填写的待升级版本。此处为示例数据，实际使用须据实修改 */
static char OneNET_version[16] = "1.0";
```

上报当前设备OneNET版本号示例如下：

```
/**
 * @brief 上报当前设备OneNET版本号
 *
 * @return
 * = 0 - 成功\n
 * = -1 - 失败
 *
 * @details 所使用的OneNET接口地址为http://ota.heclouds.com/fuse-ota/{pro_id}/{dev_name}/version\n
 * 参考网页https://open.iot.10086.cn/doc/v5/fuse/detail/1454\n
 *
 * 与平台交互过程中使用鉴权信息会过期，开发使用过程中须保证在时间过期前完成升级。安全鉴权参考
 * https://open.iot.10086.cn/doc/v5/fuse/detail/1464\n
```

```

* 因为OneNET平台会持续迭代，本demo示例在2023年6月5日可以正常使用OneNET
FOTA功能，但不能保证后续OneNET平台版本迭代后还可使用\n
* 本接口为示例代码，开发人员可参考开发
*/
static int32_t __cm_onenet_fota_report_ver(void)
{
    int32_t ret = -1;

    cm_httpclient_handle_t client = NULL;
    ret = cm_httpclient_create((const uint8_t*)"http://ota.heclouds.com", NULL, &client);

    if (CM_HTTP_RET_CODE_OK != ret || NULL == client)
    {
        cm_log_printf(0, "[OneNET] cm_httpclient_create() error!\r\n");
        return -1;
    }

    cm_httpclient_cfg_t client_cfg;
    client_cfg.ssl_enable = false;
    client_cfg.ssl_id = 0;
    client_cfg.cid = 0;
    client_cfg.conn_timeout = HTTPCLIENT_CONNECT_TIMEOUT_DEFAULT;
    client_cfg.rsp_timeout = HTTPCLIENT_WAITRSP_TIMEOUT_DEFAULT;
    client_cfg.dns_priority = 0;

    ret = cm_httpclient_set_cfg(client, client_cfg);

    char path[128] = {0};
    char header[256] = {0};
    char message[128] = {0};
    char content[64] = {0};
    unsigned char base64_tmp[128] = {0};
    int32_t et = cm_rtc_get_current_time() + 2 * 24 * 60 * 60;

    sprintf(path, "/fuse-ota");
    sprintf(path + strlen((char*)path), "/%s/%s/version", OneNET_pid, OneNET_devname);

    /* 新接口必须要上报MCU版本号，OpenCPU开发通常不涉及MCU，故s_version传入任意值 */
    sprintf(content, "\\s_version\\": \"%s\\", \\f_version\\": \"%s\\", \"MCU1.0\", OneNET_version);
    sprintf(header, "%ld\\nsha1\\nuserid/%s\\n2022-05-01", et, OneNET_userid);

    int base64_len = 0;
    mbedtls_base64_decode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
        (const unsigned char*)OneNET_userkey, strlen(OneNET_userkey));

    mbedtls_md_context_t ctx = {0};
    const mbedtls_md_info_t *info = NULL;
    mbedtls_md_init(&ctx);

    info = mbedtls_md_info_from_type(MBEDTLS_MD_SHA1);
    mbedtls_md_setup(&ctx, info, 1);
    mbedtls_md_hmac_starts(&ctx, (void *)base64_tmp, base64_len);
    mbedtls_md_hmac_update(&ctx, (void *)header, strlen(header));
    mbedtls_md_hmac_finish(&ctx, (void *)message);
    mbedtls_md_free(&ctx);

```

```

mbedtls_base64_encode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)message, 20); //SHA-1 算法输出为20字节

sprintf(header, "Authorization: version=2022-05-01&res=userid%s&set=%ld&method=sha1&sign=", "%2F",
OneNET_userid, et);

for(int i = 0; i < valueint)
{
    cJSON_Delete(root);
    cm_httpclient_specific_header_free(client);
    cm_httpclient_terminate(client);
    cm_httpclient_delete(client);
    client = NULL;

    return 0;
}
else
{
    cm_log_printf(0, "[OneNET] REPORT VERSION ERROR [%d]\r\n", code->valueint);

    cJSON_Delete(root);
    cm_httpclient_specific_header_free(client);
    cm_httpclient_terminate(client);
    cm_httpclient_delete(client);
    client = NULL;

    return -1;
}
}
}

```

检测平台是否存在升级任务示例如下：

```

/**
 * @brief 检测平台是否存在升级任务
 *
 * @return
 * = 0 - 存在升级任务 \n
 * = -1 - 失败或者不存在升级任务
 *
 * @details 所使用的OneNET接口地址为http://ota.heclouds.com/fuse-ota/{pro\_id}/{dev\_name}/check \n
 * 参考网页https://open.iot.10086.cn/doc/v5/fuse/detail/1447 \n
 *
 * 与平台交互过程中使用鉴权信息会过期，开发使用过程中须保证在时间过期前完成升级。安全鉴权参考
https://open.iot.10086.cn/doc/v5/fuse/detail/1464 \n
 * 因为OneNET平台会持续迭代，本demo示例在2023年6月5日可以正常使用OneNET
 * FOTA功能，但不能保证后续OneNET平台版本迭代后还可使用 \n
 * 本接口为示例代码，开发人员可参考开发
 */
static int32_t __cm_onenet_fota_check_upgrade_task(void)
{
    int32_t ret = -1;

    cm_httpclient_handle_t client = NULL;
    ret = cm_httpclient_create((const uint8_t*)"http://ota.heclouds.com", NULL, &client);

    if (CM_HTTP_RET_CODE_OK != ret || NULL == client)

```



```

{
    cm_log_printf(0, "[OneNET] cm_httpclient_create() error!\r\n");
    return -1;
}

cm_httpclient_cfg_t client_cfg;
client_cfg.ssl_enable = false;
client_cfg.ssl_id = 0;
client_cfg.cid = 0;
client_cfg.conn_timeout = HTTPCLIENT_CONNECT_TIMEOUT_DEFAULT;
client_cfg.rsp_timeout = HTTPCLIENT_WAITRSP_TIMEOUT_DEFAULT;
client_cfg.dns_priority = 0;

ret = cm_httpclient_set_cfg(client, client_cfg);

char path[128] = {0};
char header[256] = {0};
char message[128] = {0};
unsigned char base64_tmp[128] = {0};
int32_t et = cm_rtc_get_current_time() + 2 * 24 * 60 * 60;

sprintf(path, "/fuse-ota");
sprintf(path + strlen((char*)path), "%s/%s/check?type=1&version=%s", OneNET_pid, OneNET_devname,
OneNET_version);

sprintf(header, "%ld\nsha1\nuserid/%s\n2022-05-01", et, OneNET_userid);

int base64_len = 0;
mbedtls_base64_decode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)OneNET_userkey, strlen(OneNET_userkey));

mbedtls_md_context_t ctx = {0};
const mbedtls_md_info_t *info = NULL;
mbedtls_md_init(&ctx);

info = mbedtls_md_info_from_type(MBEDTLS_MD_SHA1);
mbedtls_md_setup(&ctx, info, 1);
mbedtls_md_hmac_starts(&ctx, (void *)base64_tmp, base64_len);
mbedtls_md_hmac_update(&ctx, (void *)header, strlen(header));
mbedtls_md_hmac_finish(&ctx, (void *)message);
mbedtls_md_free(&ctx);
mbedtls_base64_encode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)message, 20); //SHA-1 算法输出为20字节

sprintf(header, "Authorization: version=2022-05-01&res=userid%s&et=%ld&method=sha1&sign=", "%2F",
OneNET_userid, et);

for(int i = 0; i < valueint)
{
    cJSON *data = cJSON_GetObjectItem(root, "data");

    if(data == NULL)
    {
        cm_log_printf(0, "[OneNET] HTTP GET data ERROR\r\n");

        cJSON_Delete(root);
    }
}

```



```
cm_httpclient_specific_header_free(client);
cm_httpclient_terminate(client);
cm_httpclient_delete(client);
client = NULL;

return -1;
}

cJSON *tid = cJSON_GetObjectItem(data, "tid");

if(tid == NULL)
{
    cm_log_printf(0, "[OneNET] HTTP GET tid ERROR\r\n");

    cJSON_Delete(root);
    cm_httpclient_specific_header_free(client);
    cm_httpclient_terminate(client);
    cm_httpclient_delete(client);
    client = NULL;

    return -1;
}

char c_tid[16] = {0};
sprintf(c_tid, "%d", tid->valueint);

/* 将tid写入文件系统，用于记录 */
if (true == cm_fs_exist("onenet_tid.txt"))
{
    cm_fs_delete("onenet_tid.txt");
}

int32_t fd = cm_fs_open("onenet_tid.txt", CM_FS_WB);

int32_t f_len = cm_fs_write(fd, c_tid, strlen(c_tid));

if (f_len != strlen(c_tid))
{
    cm_log_printf(0, "[OneNET] FS WRITE tid ERROR\r\n");
}

cm_fs_close(fd);

cJSON_Delete(root);
cm_httpclient_specific_header_free(client);
cm_httpclient_terminate(client);
cm_httpclient_delete(client);
client = NULL;

return 0;
}

else
{
    cm_log_printf(0, "[OneNET] REPORT VERSION ERROR [%d]\r\n", code->valueint);

    cJSON_Delete(root);
```

```

cm_httpclient_specific_header_free(client);
cm_httpclient_terminate(client);
cm_httpclient_delete(client);
client = NULL;

return -1;
}
}

```

组包下载url地址，并通过cm_fota_set_url()完成配置示例如下：

```

/**
 * @brief 组包下载url地址，并通过cm_fota_set_url()完成配置
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败
 *
 * @details
 与平台交互过程中使用鉴权信息会过期，开发使用过程中须保证在时间过期前完成升级。安全鉴权参考
https://open.iot.10086.cn/doc/v5/fuse/detail/1464 \n
 * 因为onenet平台会持续迭代，本demo示例在2023年6月5日可以正常使用onenet
 fota功能，但不能保证后续onenet平台版本迭代后还可使用 \n
 *
 鉴权数据原为header中数据，因本模组平台仅可使用最小系统升级方案，不支持header配置，故需将header数据以参数形式
 添加进url中（ onenet平台支持该方式 ） \n
 * 本接口为示例代码，开发人员可参考开发
 */
static int32_t __cm_onenet_fota_set_url(void)
{
    char download_url[256] = {0};

    char c_tid[16] = {0};
    int32_t fd = cm_fs_open("onenet_tid.txt", cm_fs_rb);
    int32_t f_len = cm_fs_read(fd, c_tid, 32);
    cm_fs_close(fd);

    if (0 == f_len)
    {
        cm_log_printf(0, "[onenet] no tid\r\n");
        return -1;
    }

    int32_t et = cm_rtc_get_current_time() + 2 * 24 * 60 * 60;

    sprintf(download_url, "http://ota.heclouds.com");
    sprintf(download_url + strlen((char*)download_url), "/fuse-ota/%s/%s/%s/download", onenet_pid, onenet_devname,
        c_tid);

    /*
    鉴权数据原为header中数据，因本模组平台仅可使用最小系统升级方案，不支持header配置，故需将header数据以参数形式
    添加进url中（ onenet平台支持该方式 ） */
    char header[128] = {0};
    unsigned char base64_tmp[128] = {0};
    char message[128] = {0};
    int base64_len = 0;

```

```

sprintf(header, "%ld\nsha1\nuserid/%s\n2022-05-01", et, onenet_userid);

mbedtls_base64_decode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)onenet_userkey, strlen(onenet_userkey));

mbedtls_md_context_t ctx = {0};
const mbedtls_md_info_t *info = null;
mbedtls_md_init(&ctx);

info = mbedtls_md_info_from_type(mbedtls_md_sha1);
mbedtls_md_setup(&ctx, info, 1);
mbedtls_md_hmac_starts(&ctx, (void *)base64_tmp, base64_len);
mbedtls_md_hmac_update(&ctx, (void *)header, strlen(header));
mbedtls_md_hmac_finish(&ctx, (void *)message);
mbedtls_md_free(&ctx);
mbedtls_base64_encode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)message, 20); //sha-1 算法输出为20字节

/*
鉴权数据原为header中数据，因本模组平台仅可使用最小系统升级方案，不支持header配置，故需将header数据以参数形式
添加进url中（onenet平台支持该方式）*/
sprintf(download_url + strlen((char*)download_url), "?res=userid%s&set=%ld&method=sha1&sign=", "%2f",
onenet_userid, et);

for(int i = 0; i < base64_len; i++)
{
    if((base64_tmp[i] == '+') || (base64_tmp[i] == '/') || (base64_tmp[i] == '='))
    {
        sprintf(download_url+strlen(download_url), "%s", (base64_tmp[i] == '+')?("%2B"):(base64_tmp[i]
== '/')?("%2F"):(("%3D"))));
    }
    else
    {
        sprintf(download_url+strlen(download_url), "%c", base64_tmp[i]);
    }
}

cm_log_printf(0, "[OneNET] download_url:%s\r\n", download_url);

cm_demo_printf("[OneNET] download_url:%s\r\n", download_url);

cm_fota_res_callback_register((cm_fota_result_callback)__cm_fota_cb);

return cm_fota_set_url(download_url);
}

```

进入最小系统执行升级示例如下：

```

cm_fs_system_info_t fs_info = {0, 0};
cm_fs_getinfo(&fs_info);

/* 文件系统剩余空间为0时不建议执行升级，关键文件系统操作可能会失败 */
if (0 == fs_info.free_size)
{
    cm_log_printf(0, "insufficient space left in the file system");
    return;
}

```

```

}

cm_fota_info_t info = {0};
cm_fota_read_config(&info);

if (CM_FOTA_TYPE_HTTP_HTTPS == info.fixed_info.fota_mode)
{
    cm_log_printf(0, "HTTP server [%s]", info.fixed_info.url);
}
else
{
    cm_log_printf(0, "HTTP server error");
    return;
}

osDelay(200);

cm_fota_exec_upgrade();

```

待模组完成升级后向OneNET平台上报当前设备升级完成状态进度示例如下：

```

/**
 * @brief 上报当前设备升级完成状态
 *
 * @return
 * = 0 - 存在升级任务 \n
 * = -1 - 失败或者不存在升级任务
 *
 * @details 所使用的OneNET接口地址为http://ota.heclouds.com/fuse-ota/{pro\_id}/{dev\_name}/{tid}/status \n
 * 参考网页https://open.iot.10086.cn/doc/v5/fuse/detail/1449 \n
 * 此接口示例用于模组升级完成时向平台上报升级完成状态，不含上报其余状态逻辑 \n
 *
 * 与平台交互过程中使用鉴权信息会过期，开发使用过程中须保证在时间过期前完成升级。安全鉴权参考
https://open.iot.10086.cn/doc/v5/fuse/detail/1464 \n
 * 因为OneNET平台会持续迭代，本demo示例在2023年6月5日可以正常使用OneNET
 FOTA功能，但不能保证后续OneNET平台版本迭代后还可使用 \n
 * 本接口为示例代码，开发人员可参考开发
 */
static int32_t __cm_onenet_fota_report_progress(void)
{
    int32_t ret = -1;

    char c_tid[16] = {0};
    int32_t fd = cm_fs_open("onenet_tid.txt", CM_FS_RB);
    int32_t f_len = cm_fs_read(fd, c_tid, 32);
    cm_fs_close(fd);

    if (0 == f_len)
    {
        cm_log_printf(0, "[OneNET] no tid\r\n");
        return -1;
    }

    cm_httpclient_handle_t client = NULL;
    ret = cm_httpclient_create((const uint8_t*)"http://ota.heclouds.com", NULL, &client);

```

```

if (CM_HTTP_RET_CODE_OK != ret || NULL == client)
{
    cm_log_printf(0, "[OneNET] cm_httpclient_create() error!\r\n");
    return -1;
}

cm_httpclient_cfg_t client_cfg;
client_cfg.ssl_enable = false;
client_cfg.ssl_id = 0;
client_cfg.cid = 0;
client_cfg.conn_timeout = HTTPCLIENT_CONNECT_TIMEOUT_DEFAULT;
client_cfg.rsp_timeout = HTTPCLIENT_WAITRSP_TIMEOUT_DEFAULT;
client_cfg.dns_priority = 0;

ret = cm_httpclient_set_cfg(client, client_cfg);

char path[128] = {0};
char header[256] = {0};
char message[128] = {0};
char content[64] = "[\"step\":201]"; //升级成功，此时平台会把设备的版本号修改为任务的目标版本（设备状态变成：升级完成）
unsigned char base64_tmp[128] = {0};
int32_t et = cm_rtc_get_current_time() + 2 * 24 * 60 * 60;

sprintf(path, "/fuse-ota");
sprintf(path + strlen((char*)path), "%s/%s/%s/status", OneNET_pid, OneNET_devname, c_tid);
sprintf(header, "%ld\\nsha1\\nuserid\\n2022-05-01", et, OneNET_userid);

int base64_len = 0;
mbedtls_base64_decode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)OneNET_userkey, strlen(OneNET_userkey));

mbedtls_md_context_t ctx = {0};
const mbedtls_md_info_t *info = NULL;
mbedtls_md_init(&ctx);

info = mbedtls_md_info_from_type(MBEDTLS_MD_SHA1);
mbedtls_md_setup(&ctx, info, 1);
mbedtls_md_hmac_starts(&ctx, (void *)base64_tmp, base64_len);
mbedtls_md_hmac_update(&ctx, (void *)header, strlen(header));
mbedtls_md_hmac_finish(&ctx, (void *)message);
mbedtls_md_free(&ctx);
mbedtls_base64_encode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)message, 20); //SHA-1 算法输出为20字节

sprintf(header, "Authorization: version=2022-05-01&res=userid%s&et=%ld&method=sha1&sign=", "%2F",
OneNET_userid, et);

for(int i = 0; i < valueint)
{
    cJSON_Delete(root);
    cm_httpclient_specific_header_free(client);
    cm_httpclient_terminate(client);
    cm_httpclient_delete(client);
    client = NULL;
}

```

```

    return 0;
}
else
{
    cm_log_printf(0, "[OneNET] REPORT VERSION ERROR [%d]\r\n", code->valueint);

    cJSON_Delete(root);
    cm_httpclient_specific_header_free(client);
    cm_httpclient_terminate(client);
    cm_httpclient_delete(client);
    client = NULL;

    return -1;
}
}

```

全系统差分升级方案或APP整包升级方案

1. 全系统差分升级方案或APP整包升级方案下，HTTP方式下载升级包数据并升级（本文档仅摘录部分代码，完整代码参考cm_demo_fota.c中源码）。

OTA初始化、下载升级包及升级示例如下：

```

/**
 * OTA功能调试使用示例,适用于文件系统OTA升级,包含全差分升级以及APP整包单独升级
 * CM:OTA:INIT //OTA初始化
 * CM:OTA:WRITE_PACKAGE //OTA 下载/写入固件升级包至文件系统
 * CM:OTA:UPGRADE //校验并升级，须在将OTA升级包写入文件系统后运行
 * CM:OTA:ERASE //文件系统中擦除OTA升级包
 */
void cm_test_ota(unsigned char **cmd,int len)
{
    unsigned char operation[20] = {0};
    sprintf((char *)operation, "%s", cmd[2]);

    if (0 == strcmp((const char *)operation, "INIT")) //初始化
    {
        cm_demo_printf("ota update success\n");
        cm_ota_init();
    }
    else if (0 == strcmp((const char *)operation, "WRITE_PACKAGE")) //文件系统OTA方式升级
    {
        const char *server_url = "http://xxx.com:8080"; //仅作为示例，url不可使用
        const char *url_path = "/download/system_patch.bin"; //仅作为示例，url path不可使用
        if(s_ota_http_handle == NULL)
        {
            cm_httpclient_create((const uint8_t *)server_url, __cm_ota_download_cb, &s_ota_http_handle); //初始化并设置回调函数
        }

        if(s_ota_http_handle == NULL) //初始化失败
        {
            cm_demo_printf("http handhle init fail\n");
            return;
        }
    }
}

```

```

cm_httpclient_request_start(s_ota_http_handle, HTTPCLIENT_REQUEST_GET, (const uint8_t*)url_path, false, 0);
}
else if (0 == strcmp((const char *)operation, "UPGRADE")) //启动升级
{
    cm_ota_upgrade();//开始升级，包括升级包校验、复位等操作
}
else if (0 == strcmp((const char *)operation, "ERASE")) //从文件系统删除升级包
{
    cm_ota_firmware_erase();
}
else if (0 == strcmp((const char *)operation, "RES_SIZE")) //文件系统剩余空间大小查询
{
    cm_demo_printf("file system res size[%d]\n", cm_ota_get_capacity());
}
else
{
    cm_demo_printf("[OTA] Illegal operation\n");
}
return;
}

```

回调函数中将HTTP下载的升级包数据写入模组示例如下：

```

/**
 * \brief http下载回调函数
 *
 * \param [in] client_handle http句柄
 * \param [in] event http回调类型
 * \param [in] param http数据
 *
 * \return None
 *
 * \details 此示例下，http content内容即为差分包内容
 */
static void __cm_ota_download_cb(cm_httpclient_handle_t client_handle, cm_httpclient_callback_event_e
event, void *param)
{
    (void)client_handle;
    int ret = 0;

    if(param == NULL || event != CM_HTTP_CALLBACK_EVENT_RSP_CONTENT_IND)
    {
        return;
    }

    /*写入升级包*/
    cm_httpclient_callback_rsp_content_param_t *ota_pkg = (cm_httpclient_callback_rsp_content_param_t *)param;

    if(ota_pkg->current_len == ota_pkg->sum_len)
    {
        cm_log_printf(0, "set total\n");
        cm_ota_set_otasize(ota_pkg->total_len);
    }

    ret = cm_ota_firmware_write((const char *)ota_pkg->response_content, ota_pkg->current_len);
    if(ret)

```



```

{
    return;
}
cm_log_printf(0, "Writed: %d/%d - %d%%\n",
/*ota_pkg->sum_len*/cm_ota_get_written_size(),
ota_pkg->total_len, ota_pkg->sum_len * 100 / ota_pkg->total_len);

if(ota_pkg->sum_len >= ota_pkg->total_len) //下载完成
{
    if(ota_pkg->sum_len > cm_ota_get_written_size())
    {
        cm_demo_printf("download ota pkg err!\n");
    }
    else
    {
        cm_demo_printf("download ota pkg complete!\n");
    }
}
}
}

```

2. 全系统差分升级方案或APP整包升级方案下，OneNET FOTA（HTTP协议）下载升级包数据并升级（本文档仅摘录部分代码，完整代码参考cm_demo_fota.c中源码）。

该方案下，除下载和写入升级包至模组操作与“最小系统升级方案下，OneNET FOTA（HTTP协议）下载升级包数据并升级”不同外，其余操作相同。

模组从OneNET平台下载升级包请求示例如下：

```

/**
 * @brief 从OneNET平台下载升级包并写入模组
 *
 * @return
 * = 0 - 成功 \n
 * < 0 - 失败
 *
 * @details 因为onenet平台会持续迭代，本demo示例在2023年8月2日可以正常使用onenet
fota功能，但不能保证后续onenet平台版本迭代后还可使用 \n
 * 本接口为示例代码，开发人员可参考开发
 */
static int32_t __cm_onenet_fota_download(void)
{
    char download_url[64] = {0};
    char download_path[96] = {0};

    char c_tid[16] = {0};
    int32_t fd = cm_fs_open("onenet_tid.txt", cm_fs_rb);
    int32_t f_len = cm_fs_read(fd, c_tid, 32);
    cm_fs_close(fd);

    if (0 == f_len)
    {
        cm_log_printf(0, "[onenet] no tid!\n");
        return -1;
    }

    int32_t et = cm_rtc_get_current_time() + 2 * 24 * 60 * 60;

```



```

sprintf(download_url, "http://ota.heclouds.com");
sprintf(download_path, "/fuse-ota/%s/%s/%s/download", onenet_pid, onenet_devname, c_tid);

unsigned char base64_tmp[128] = {0};
char message[128] = {0};
int base64_len = 0;

memset(oneenet_download_header, 0, 256);
sprintf(oneenet_download_header, "%ld\\nsha1\\nuserid/%s\\n2022-05-01", et, onenet_userid);

mbedtls_base64_decode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)onenet_userkey, strlen(oneenet_userkey));

mbedtls_md_context_t ctx = {0};
const mbedtls_md_info_t *info = null;
mbedtls_md_init(&ctx);

info = mbedtls_md_info_from_type(mbedtls_md_sha1);
mbedtls_md_setup(&ctx, info, 1);
mbedtls_md_hmac_starts(&ctx, (void *)base64_tmp, base64_len);
mbedtls_md_hmac_update(&ctx, (void *)onenet_download_header, strlen(oneenet_download_header));
mbedtls_md_hmac_finish(&ctx, (void *)message);
mbedtls_md_free(&ctx);
mbedtls_base64_encode((unsigned char*)base64_tmp, 128, (unsigned int*)&base64_len,
(const unsigned char*)message, 20); //sha-1 算法输出为20字节

sprintf(oneenet_download_header, "authorization:
version=2022-05-01&res=userid%s&et=%ld&method=sha1&sign=", "%2F", onenet_userid, et);

for(int i = 0; i < base64_len; i++)
{
    if((base64_tmp[i] == '+') || (base64_tmp[i] == '/') || (base64_tmp[i] == '='))
    {
        sprintf(OneNET_download_header+strlen(OneNET_download_header), "%s", (base64_tmp[i]
== '+' ? ("%2B") : ((base64_tmp[i] == '/') ? ("%2F") : ("%3D"))));
    }
    else
    {
        sprintf(OneNET_download_header+strlen(OneNET_download_header), "%c", base64_tmp[i]);
    }
}

if(s_ota_http_handle == NULL)
{
    cm_httpclient_create((const uint8_t*)download_url, __cm_ota_onenet_download_cb, &s_ota_http_handle); //初始
化并设置回调函数
}

if(s_ota_http_handle == NULL) //初始化失败
{
    cm_demo_printf("http handle init fail\n");
    return -1;
}

```

```

cm_httpclient_ret_code_e ret = cm_httpclient_specific_header_set(s_ota_http_handle, (uint8_t
*)OneNET_download_header, strlen(OneNET_download_header));

ret = cm_httpclient_request_start(s_ota_http_handle, HTTPCLIENT_REQUEST_GET, (const uint8_t*)download_path,
false, 0);

if (CM_HTTP_RET_CODE_OK == ret)
{
    return 0;
}
else
{
    return -1;
}
}

```

回调函数中将HTTP下载的升级包数据写入模组示例如下：

```

static void __cm_ota_onenet_download_cb(cm_httpclient_handle_t client_handle, cm_httpclient_callback_event_e
event, void *param)
{
    (void)client_handle;

    cm_demo_printf("__cm_ota_onenet_download_cb() event %d\n", event);

    if (CM_HTTP_CALLBACK_EVENT_RSP_CONTENT_IND == event)
    {
        cm_httpclient_callback_rsp_content_param_t *ota_pkg = (cm_httpclient_callback_rsp_content_param_t *)param;

        cm_demo_printf("__cm_ota_onenet_download_cb() current_len %d, sum_len %d, total_len %d\n",
ota_pkg->current_len, ota_pkg->sum_len, ota_pkg->total_len);

        if(ota_pkg->current_len == ota_pkg->sum_len)
        {
            cm_ota_set_otasize(ota_pkg->total_len);
        }

        int32_t ret = cm_ota_firmware_write((const char*)ota_pkg->response_content, ota_pkg->current_len);

        if(ret)
        {
            return;
        }

        cm_log_printf(0, "Writed: %d/%d - %d%%\n",
/*ota_pkg->sum_len*/cm_ota_get_written_size(),
ota_pkg->total_len, ota_pkg->sum_len * 100 / ota_pkg->total_len);

        if(ota_pkg->sum_len >= ota_pkg->total_len) //下载完成
        {
            if(ota_pkg->sum_len > cm_ota_get_written_size())
            {
                cm_demo_printf("download ota pkg err!\n");
            }
            else
            {

```

```
cm_demo_printf("download ota pkg complete!\n");
}
}
}
else if (CM_HTTP_CALLBACK_EVENT_RSP_HEADER_IND == event)
{
cm_httpclient_callback_rsp_header_param_t *header = (cm_httpclient_callback_rsp_header_param_t *)param;
//cm_demo_printf("__cm_ota_onenet_download_cb() response_header_len %d\n", header->response_header_len);
cm_demo_printf("__cm_ota_onenet_download_cb() header: %s\n", header->response_header);
}
}
```

Note:

使用OneNET FOTA功能时有关OneNET平台侧的创建产品、设备、升级任务等操作请参考OneNET门户网站的使用文档或咨询OneNET平台的FAE；

OneNET平台产品会不断迭代，本示例开发的代码仅为参考，无法保证适配于众多的OneNET平台版本；

有关OneNET FOTA功能终端侧软件开发指导文档请参考OneNET门户网站的使用文档或咨询OneNET平台的FAE。



5. 编程设计注意

固件升级功能在使用时应注意以下几点。

- 最小系统升级方案下，HTTP升级方式，用户需使用cm_fota_set_url()接口配置HTTP服务器信息；
- 最小系统升级方案下，下载信息配置完成并调用cm_fota_exec_upgrade()接口进入最小系统后，模组会自行下载升级包文件用于最小系统和非最小系统的升级，期间模组会自动重启4次；
- 最小系统升级方案下，升级服务器须支持断点续传功能；
- 最小系统升级方案下，由于芯片平台特性，在固件升级失败时模组无法自动回退到升级前的固件版本，用户批量升级模组固件前务必在开发调试阶段确认固件升级功能可正常使用；
- 最小系统升级方案下，下载信息配置时会进行文件系统操作，若文件系统剩余空间较小配置url可能会失败。建议使用fota功能的用户在文件系统中至少留有4096字节剩余空间（cm_fs_getinfo()接口可查询）；
- APP整包升级方案下，务必保证当前版本和目标升级版本中的系统固件system.img一致（同一SDK版本中同一子型号模组编译得到的版本中system.img一致），仅APP固件user_app.bin不同；
- 全系统差分升级方案和APP整包升级方案下，升级包保存在模组的文件系统中，即升级包会占用文件系统空间；
- 全系统差分升级方案和APP整包升级方案下，调用cm_ota_firmware_write()接口前需先调用cm_ota_set_otasize()接口配置升级包大小；
- 模组升级过程中不能断电。

6. 附录

Table 2. 缩略语

缩写	英文全称	中文解释
API	Application Programming Interface	应用程序编程接口